



Scuola Arti e Mestieri Trevano

Sezione informatica



Implementazione di Nebula in Swift

Titolo del progetto: Implementazione di Nebula in Swift
Alunno/a: Linda Bytyqi
Classe: I3AC
Anno scolastico: 2025 - 2026
Docente responsabile: Ingrid Cereda

1 Sommario

1	Sommario	2
2	Indice delle figure	4
3	Introduzione	5
3.1	Informazioni sul progetto	5
3.2	Scopo	5
4	Analisi	6
4.1	Analisi del dominio	6
4.2	Analisi e specifica dei requisiti	6
4.2.1	Requisiti funzionali	6
4.2.2	Requisiti non funzionali	8
4.2.3	Spiegazione elementi tabella dei requisiti	8
4.2.4	Elementi della tabella:	8
4.3	Use case	10
4.4	Pianificazione	11
4.4.1	Diagramma di Gantt	12
4.4.2	Periodo	12
4.4.3	Struttura	12
4.4.4	Conclusione	12
4.5	Analisi dei mezzi	13
4.5.1	Software	13
4.5.2	Hardware	13
5	Progettazione	14
5.1	Design dell'architettura del sistema	14
5.2	Design dei dati e database	15
5.2.1	NebulaUML	15
5.3	Design delle interfacce	16
5.3.1	Pagina principale	16
5.3.2	Sistema Solare	17
5.3.3	Informazioni sui pianeti	18
5.3.4	Informazioni sulla Luna	19
5.3.5	L'immagine del giorno	20
5.3.6	Galassia	21
5.4	Design procedurale	22
6	Implementazione	24
6.1	Creazione applicativo	24
6.2	ContentView.swift	24
6.2.1	Codice significativo	24
6.2.2	Interfaccia	25
6.3	HomeView.swift	26
6.3.1	Codice significativo	26
6.3.2	Interfaccia	27
6.4	InfoView.swift	28
6.4.1	Codice significativo	28
6.4.2	Interfaccia	29
6.5	MyApp.swift	30
6.6	Planet.swift	30
6.6.1	Codice significativo	30
6.7	PlanetData.swift	31
6.7.1	Codice significativo	31
6.8	PlanetDetailView.swift	32
6.8.1	Codice significativo	32
6.8.2	Interfaccia	33
6.9	PlanetView.swift	34
6.9.1	Codice significativo	34
7	Test	35
7.1	Protocollo di test	35

7.2	Risultati test.....	38
7.3	Mancanze/limitazioni conosciute.....	38
8	Consuntivo.....	39
8.1	Analisi.....	40
8.2	Progettazione	40
8.3	Implementazione	40
8.4	Documentazione	40
8.5	Differenze	40
8.6	Conclusione.....	41
9	Conclusioni	42
9.1	Sviluppi futuri.....	42
9.2	Considerazioni personali.....	42
10	Glossario.....	43
11	Bibliografia	45
11.1	Sitografia.....	45
12	Allegati	45

2 Indice delle figure

Figura 1 - Use Case.....	10
Figura 2 - Gantt Pianificazione	11
Figura 3 - Diagramma UML	15
Figura 4 - Pagina principale Mockup	16
Figura 5 - Sistema Solare Mockup	17
Figura 6 - Informazioni sui pianeti Mockup	18
Figura 7 - Informazioni Luna Mockup	19
Figura 8 - APOD Mockup.....	20
Figura 9 - Galassia Mockup.....	21
Figura 10 - Diagramma di flusso	22
Figura 11 - NavigationLink.....	24
Figura 12 - Pagina iniziale	25
Figura 13 – Accesso a PlanetDetailView.....	26
Figura 14 - Sistema Solare	27
Figura 15 - API.....	28
Figura 16 - Interfaccia APOD	29
Figura 17 – Struttura JSON	30
Figura 18 - Gestione errori.....	31
Figura 19 – Gestione dei video.....	32
Figura 20 - Informazioni sul pianeta	33
Figura 21 - Rotazione	34
Figura 22 - Risultato dei test.....	38
Figura 23 - Gantt Consuntivo	39



3 Introduzione

3.1 Informazioni sul progetto

- **Titolo:** Implementazione di Nebula in Swift;
- **Allieva:** Linda Bytyqi;
- **Sede:** CPT Trevano;
- **Docente:** Ingrid Cereda;
- **Modulo:** M306 - SAMT Progetti;
- **Periodo:** 05.09.2025 – 19.12.2025

3.2 Scopo

Lo scopo del progetto Nebula è quello di realizzare un'applicazione interattiva e didattica, sviluppata in Swift Playgrounds, che consenta all'utente di esplorare il Sistema Solare in modo semplice, coinvolgente e visivamente accattivante.

L'app è pensata principalmente per studenti e appassionati di astronomia, che desiderano approfondire la conoscenza dei pianeti e dei corpi celesti in maniera intuitiva, senza la necessità di utilizzare strumenti complessi o professionali.

Attraverso animazioni dinamiche e un'interfaccia accessibile, il progetto intende avvicinare le persone al mondo dell'astronomia, trasformando l'apprendimento in un'esperienza interattiva.

Ogni pianeta potrà essere selezionato per visualizzare informazioni, immagini e una curiosità del giorno.

L'obiettivo finale è quello di unire educazione e tecnologia, offrendo un prodotto che:

- stimoli la curiosità e l'interesse scientifico;
- renda la conoscenza del Sistema Solare più comprensibile e interattiva;
- favorisca un apprendimento visivo attraverso immagini e animazioni;
- mostri come il linguaggio Swift possa essere utilizzato per creare applicazioni multimediali anche in ambito educativo.

Inoltre, il progetto mira a dimostrare l'utilità di Swift Playgrounds come strumento non solo per l'apprendimento della programmazione, ma anche per la realizzazione di piccoli prodotti digitali completi e interattivi.

In sintesi, Nebula ha come scopo principale quello di divulgare la conoscenza astronomica in modo moderno e accessibile, creando un ponte tra scienza e tecnologia attraverso un applicativo semplice ma di grande impatto visivo.

4 Analisi

4.1 Analisi del dominio

Il progetto nasce dall'esigenza di creare un'applicazione didattica e divulgativa che consente di esplorare il Sistema Solare in modo interattivo. L'astronomia è da sempre un campo affascinante, ma purtroppo spesso e volentieri le risorse disponibili sono adatte per un pubblico esperto, ciò le rende più complesse da comprendere per chi è passionato ma non se ne intende molto. L'obiettivo quindi è quello di proporre un prodotto semplice, intuitivo e coinvolgente che permette agli appassionati di informarsi sull'astronomia. Il prodotto è pensato per essere utilizzato principalmente in ambito informativo da chi ne è appassionato oppure in contesto scolastico. Al momento esistono app avanzate come Sky Map, ma c'è bisogno di un'ulteriore conoscenza per comprenderle poiché sono abbastanza complesse. Gli utenti principali sono studenti, appassionati di astronomia principianti e professori, l'obiettivo è quello di avere un'interfaccia semplice che permetta di soddisfare i bisogni principali degli utenti perciò non hanno bisogno di competenze particolari per utilizzare l'applicativo.

4.2 Analisi e specifica dei requisiti

In questo capitolo si descrivono le funzionalità che il sistema dovrà offrire e le caratteristiche qualitative richieste, in modo che lo sviluppo possa basarsi su una base chiara e condivisa.

4.2.1 Requisiti funzionali

ID: REQ-01	
Nome	Schermata Home
Priorità	1
Versione	1.0
Note	Interfaccia iniziale, una volta cliccato il bottone si entra nella pagina in cui si trova il Sistema Solare.
ID: REQ-02	
Nome	Sistema Solare
Priorità	1
Versione	1.0
Note	In questa pagina ci sarà il sole in mezzo e i pianeti che ruotano attorno ad esso.
Sotto requisiti	
001	Si necessita il corretto funzionamento del Req-01.
ID: REQ-03	
Nome	Ingrandimento della pagina
Priorità	1
Versione	1.0



Note	Nella pagina che rappresenta il Sistema Solare sarà possibile ingrandire la pagina.
Sotto requisiti	
001	Si necessita il corretto funzionamento del Req-02.
ID: REQ-04	
Nome	Rimpicciolimento della pagina
Priorità	1
Versione	1.0
Note	Nella pagina che rappresenta il Sistema Solare sarà possibile rimpicciolire la pagina e vedere la Via Lattea.
Sotto requisiti	
001	Si necessita il corretto funzionamento del Req-02.
ID: REQ-05	
Nome	Scelta del pianeta
Priorità	1
Versione	1.0
Note	Se vengono premuti i pianeti si entrerà nella pagina delle informazioni riguardanti essi.
Sotto requisiti	
001	Si necessita il corretto funzionamento del Req-02.
ID: REQ-06	
Nome	Informazioni pianeta
Priorità	1
Versione	1.0
Note	Lettura sulle informazioni riguardante il pianeta, inoltre ci sarà un video esplicativo e un'immagine che rappresenta il pianeta.
Sotto requisiti	
001	Informazioni corrette.
002	File JSON.
003	Si necessita il corretto funzionamento del Req-03.
ID: REQ-07	
Nome	Luna
Priorità	1

Versione	1.0
Note	Premendo sulla Luna si avrà il calendario lunare e le sue varie fasi.
Sotto requisiti	
001	Si necessita il corretto funzionamento del Req-02
ID: REQ-08	
Nome	Curiosità del giorno
Priorità	1
Versione	1.0
Note	Premendo un bottone si aprirà una schermata che mostrerà la curiosità del giorno implementando la foto scattata dalla NASA proprio quel giorno.
Sotto requisiti	
001	Corretto funzionamento dell'API.
002	Corretto funzionamento della navigazione tra le pagine.

4.2.2 Requisiti non funzionali

Questi requisiti riguardano le caratteristiche di qualità, vincoli e condizioni generali che il sistema deve rispettare.

- **Usabilità:** l'interfaccia deve essere semplice e immediata, adatta a utenti con poca esperienza.
- **Prestazioni:** le animazioni devono essere fluide (es. rotazione dei pianeti senza scatti) e i tempi di risposta rapidi.
- **Compatibilità:** l'app deve funzionare nell'ambiente di sviluppo scelto (Swift Playgrounds su iPad).
- **Affidabilità e correttezza dei dati:** le informazioni mostrate devono derivare da fonti affidabili (es. NASA) e il caricamento dei dati JSON deve avvenire senza errori.
- **Estetica e design:** tema spaziale coerente (sfondo scuro, stelle), testo leggibile con buon contrasto.
- **Scalabilità:** il sistema deve permettere in futuro l'aggiunta di nuovi pianeti o curiosità.

4.2.3 Spiegazione elementi tabella dei requisiti:

La tabella dei requisiti rappresenta un riepilogo strutturato di tutte le caratteristiche che il sistema deve possedere.

Ogni requisito è identificato da un codice univoco e descritto attraverso una serie di campi che ne specificano la natura, la priorità, la versione e le eventuali relazioni con altri requisiti.

4.2.4 Elementi della tabella:

ID:

È l'identificativo univoco del requisito.

Serve per distinguere in modo chiaro ogni requisito dagli altri e per poterlo facilmente richiamare all'interno della documentazione.

Generalmente viene espresso nella forma *REQ-XX* (es. REQ-01 per il primo requisito).

**Nome:**

Indica una breve descrizione del requisito, ossia il suo titolo sintetico che riassume in poche parole la funzionalità o la caratteristica richiesta.

Ad esempio: *“Schermata iniziale”*.

Priorità:

Specifica l'importanza del requisito all'interno del progetto.

La priorità viene definita in accordo con il committente e può avere diversi livelli (ad esempio:

- **1 = alta** → requisito fondamentale, necessario per il funzionamento del sistema;
- **2 = media** → requisito importante ma non essenziale;
- **3 = bassa** → requisito secondario o opzionale).

La priorità aiuta a stabilire quali requisiti devono essere implementati per primi.

Versione:

Indica la versione del requisito, utile per tenere traccia di eventuali modifiche nel tempo.

Ogni aggiornamento o revisione comporta un incremento della versione (es. 1.0 → 1.1 → 2.0).

Nella documentazione principale viene mostrata solo l'ultima versione, mentre le precedenti vengono archiviate nei diari di progetto.

Note:

Campo dedicato a eventuali osservazioni aggiuntive, specifiche tecniche, riferimenti a fonti esterne o collegamenti con altri requisiti.

Ad esempio: *“Si necessita il corretto funzionamento del REQ-01”* oppure *“Immagini fornite dalla NASA”*.

Sotto-requisiti:

Elenca gli elementi più specifici o le sotto-funzionalità che compongono il requisito principale.

Servono per descrivere in modo più dettagliato come il requisito sarà realizzato o quali condizioni devono essere soddisfatte affinché il requisito sia considerato completo.

Ad esempio:

- REQ-03 (Sistema Solare interattivo) può avere come sotto-requisiti:
- REQ-03.001: “I pianeti devono ruotare intorno al Sole”;
- REQ-03.002: “Deve essere possibile zoomare”.



4.3 Use case

Lo Use Case (o Caso d'Uso) rappresenta uno strumento di analisi che serve a descrivere come gli attori interagiscono con il sistema per raggiungere un determinato obiettivo.

In altre parole, uno Use Case mostra in modo chiaro che cosa fa il sistema (non come lo fa) e in che modo gli utenti o le altre entità esterne lo utilizzano.

Nel contesto del progetto *Nebula*, l'attore principale è:

- **Utente** → colui che utilizza l'app per esplorare il sistema solare e leggere le informazioni.

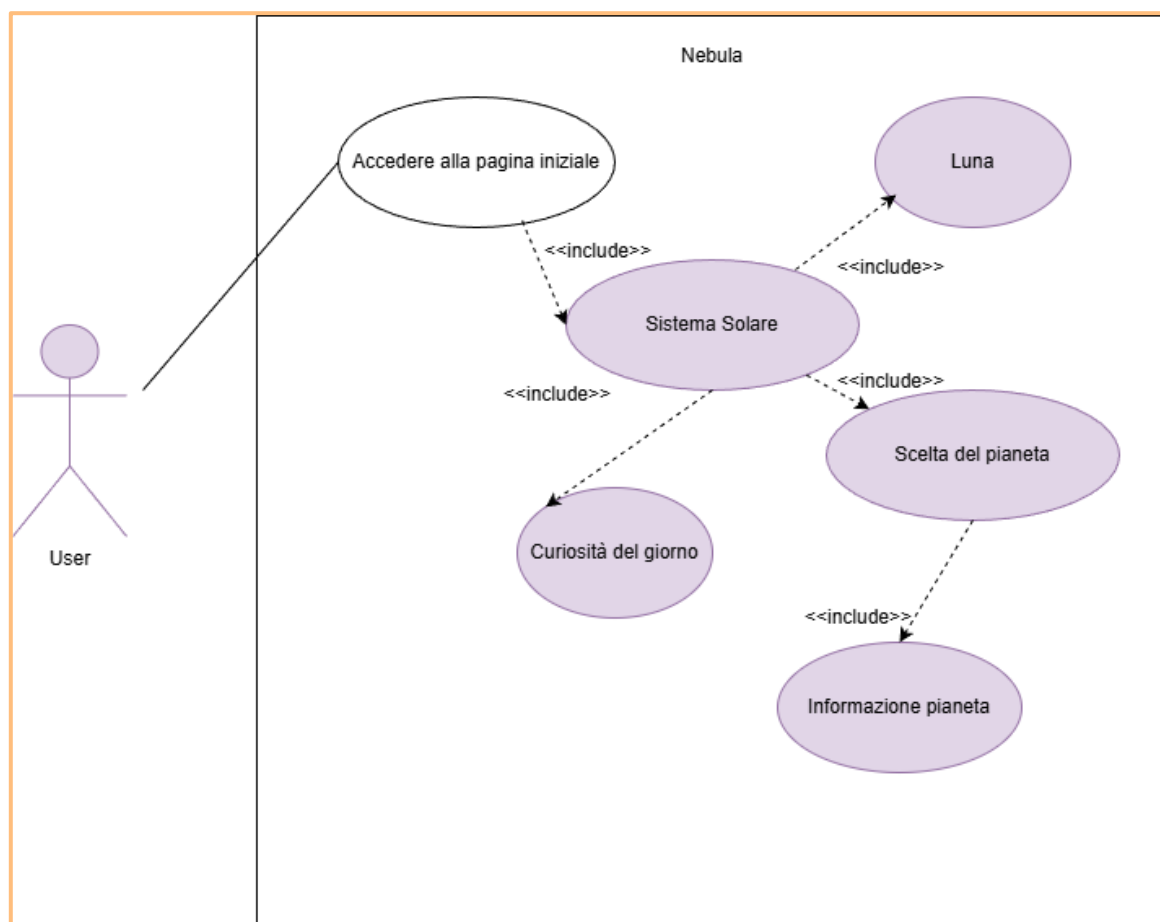


Figura 1 - Use Case



4.4 Pianificazione

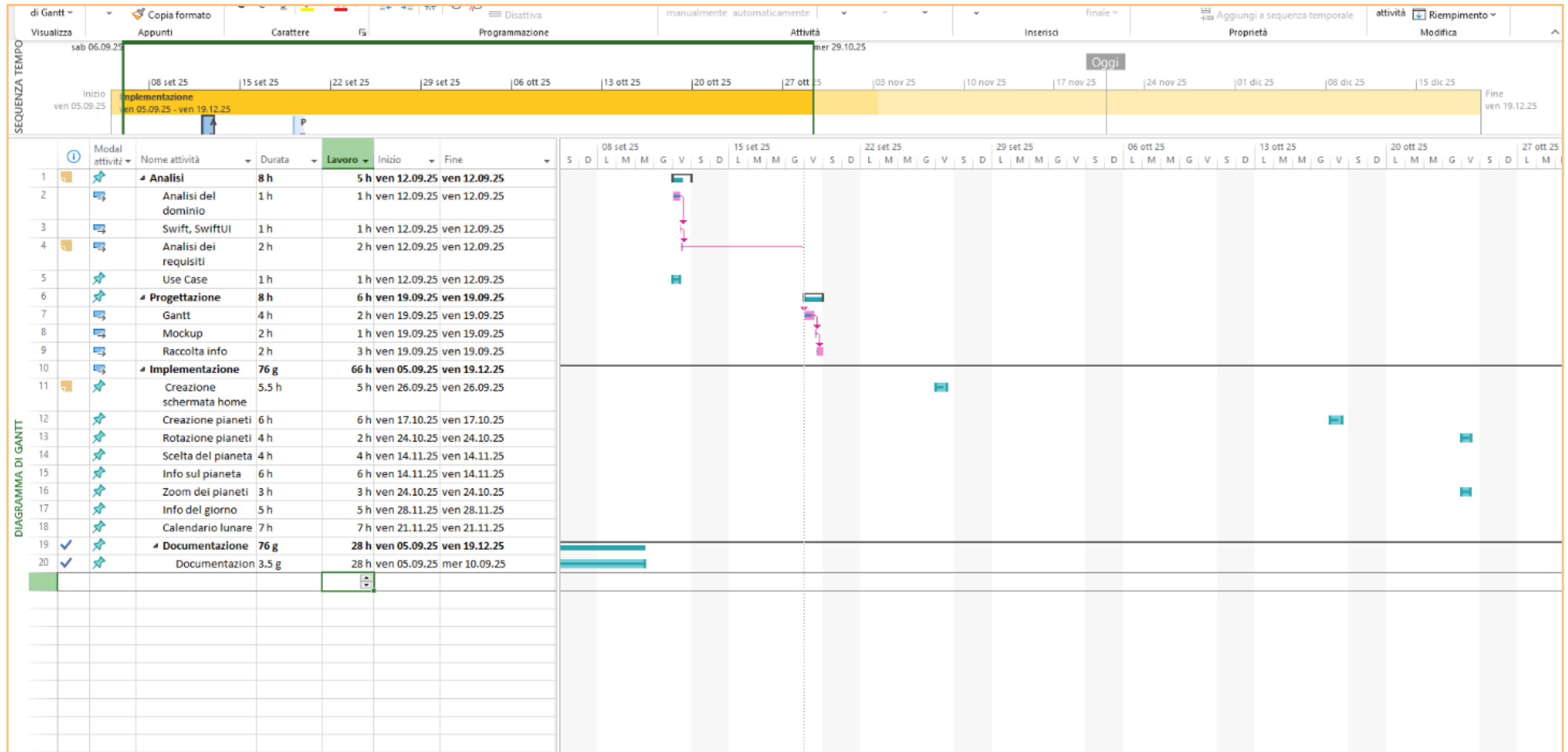


Figura 2 - Gantt Pianificazione



4.4.1 Diagramma di Gantt

Il diagramma di Gantt rappresenta la pianificazione temporale delle attività del progetto, evidenziandone la suddivisione in fasi, la durata, le dipendenze e l'arco temporale complessivo di sviluppo.

4.4.2 Periodo

Il progetto si sviluppa nel periodo compreso tra il 05/09/2025 e il 19/12/2025, come indicato dalla sequenza temporale principale. Il diagramma consente di monitorare l'avanzamento delle attività nel tempo e di individuare eventuali sovrapposizioni o vincoli tra le fasi.

4.4.3 Struttura

Il lavoro è organizzato in quattro fasi principali:

- **Analisi:** ha una durata complessiva di 8 ore ed è concentrata nella giornata del 12/09/2025. Comprende le seguenti attività:

- Analisi del dominio
- Analisi dei requisiti
- Studio delle tecnologie (Swift, SwiftUI)
- Definizione degli Use case

Questa fase ha l'obiettivo di comprendere il contesto applicativo, definire i requisiti funzionali e tecnologici e fornire una base solida per la progettazione.

- **Progettazione:** dura complessivamente 8 ore e si svolge il 19/09/2025. Include:

- Creazione del diagramma di Gantt
- Realizzazione dei Mockup
- Raccolta delle informazioni necessarie

Questa fase consente di strutturare l'architettura dell'applicazione e di definire l'interfaccia utente prima dell'implementazione.

- **Implementazione:** questa fase è la più estesa del progetto, con una durata complessiva di 76 giorni (66 ore di lavoro distribuite nel tempo), dal 05/09/2025 al 19/12/2025. Le principali funzionalità sviluppate sono:

- Creazione della schermata home
- Creazione e gestione dei pianeti
- Rotazione e zoom dei pianeti
- Selezione e visualizzazione delle informazioni sui pianeti
- Visualizzazione delle informazioni del giorno
- Implementazione del calendario lunare

Le attività sono distribuite nel tempo e organizzate in modo sequenziale, in base alle dipendenze funzionali tra le varie componenti dell'applicazione.

- **Documentazione:** si sviluppa parallelamente alle altre fasi, con una durata complessiva di 76 giorni (28 ore di lavoro), dal 05/09/2025 al 19/12/2025. Questa scelta consente di mantenere la documentazione costantemente aggiornata durante l'intero ciclo di sviluppo del progetto.
- **Dipendenze e controllo:** nel diagramma sono visibili le dipendenze tra le attività, rappresentate dalle frecce, che indicano l'ordine di esecuzione e i vincoli temporali. La linea verticale "Oggi" permette di confrontare la pianificazione con lo stato di avanzamento reale del progetto.

4.4.4 Conclusione

Il diagramma di Gantt fornisce una visione chiara e strutturata dell'intero progetto, facilitando la pianificazione, il monitoraggio delle attività e il rispetto delle scadenze. È uno strumento fondamentale per garantire una gestione efficace del tempo e delle risorse.

4.5 Analisi dei mezzi

Swift e SwiftUI è il framework principale per la realizzazione di applicazioni per dispositivi Apple, tra cui iPhone, iPad, Mac, Apple Watch e Apple TV. Swift è un linguaggio di programmazione moderno, sicuro e veloce, creato da Apple per lo sviluppo di app su tutti i suoi dispositivi. SwiftUI è il framework dichiarativo di Apple per costruire interfacce grafiche, introdotto nel 2019, che semplifica la progettazione di layout, animazioni e navigazione tra schermate.

SwiftUI permette di realizzare interfacce utilizzando una sola codebase e gestisce automaticamente l'aggiornamento della UI quando cambiano i dati, riducendo il codice necessario rispetto a framework tradizionali come UIKit. Inoltre, consente di personalizzare stili e comportamenti specifici per ciascun dispositivo Apple, mantenendo una coerenza grafica tra piattaforme diverse.

Questo framework è particolarmente indicato per chi vuole sviluppare applicazioni native per l'ecosistema Apple utilizzando Swift, con un approccio moderno e intuitivo, e per chi desidera integrare animazioni, gesti e dati in modo semplice.

Per il progetto, ho utilizzato Swift Playground 4.6.4, con Swift 5.9 come ambiente di sviluppo, con SwiftUI come framework per la creazione dell'interfaccia grafica.

4.5.1 Software

- Swift Playground
 - Versione: 4.6.4

4.5.2 Hardware

- Ipad:
 - Marca: Apple
 - Modello: Ipad 10th Generation 2022
 - IpadOS versione: 26.1
 - Capacità: 64GB
 - Memoria: 4GB
- Keyboard Amazon
 - Marca: GAOJIE



5 Progettazione

Questo capitolo descrive le scelte tecniche e progettuali che guidano la realizzazione dell'applicazione *Nebula*. Una corretta progettazione permette di evitare fraintendimenti durante lo sviluppo e garantisce coerenza tra requisiti, struttura del codice e interfaccia utente.

5.1 Design dell'architettura del sistema

Il sistema è composto dai seguenti moduli:

- **ContentView**
Mostra:
 - Sfondo stellato
 - Scritta di benvenuto
 - Bottone da premere
- **HomeView**
Mostra:
 - Sfondo stellato
 - Sole fisso
 - Orbite dei pianeti
 - Pianeti animati che orbitano
 - Luna fissa
 - Accesso alle pagine informative dei pianeti
 - Gestione dello zoom/zoom out
- **InfoView**
Si occupa della:
 - Gestione API
- **MyApp**
Mostra:
 - Mostra da dove parte l'app e quale schermata mostra quando si avvia
- **Planet**
Si occupa di:
 - Gestire il file JSON
- **PlanetDetailView**
Pagina interna che contiene:
 - Immagine del pianeta
 - Testo descrittivo
 - Video MP4 riproducibile con AVKit
- **PlanetData**
Si occupa di:
 - Gestire gli errori durante il caricamento del file JSON
- **PlanetView**
Si occupa del:
 - Componente per mostrare un pianeta animato lungo la sua orbita.

Flussi di informazione

Ingresso:

- Selezione dell'utente (tap sul pianeta)
- Playback dei video
- Caricamento delle immagini

Elaborazione:

- Calcolo animazione orbitale
- Rendering grafico
- Navigazione verso la pagina del pianeta

Uscita:

- Dettagli del pianeta
- Video avviabile
- Animazioni visive



5.2 Design dei dati e database

Il sistema non utilizza un database relazionale tradizionale. I dati necessari al funzionamento dell'applicazione vengono gestiti tramite strutture dati interne e tramite un file JSON utilizzato per memorizzare le informazioni relative ai pianeti.

5.2.1 NebulaUML

Il diagramma UML presentato rappresenta il modello dei dati utilizzato dall'applicazione. Poiché il sistema non si appoggia a un database relazionale, la struttura dati è progettata seguendo un approccio orientato agli oggetti, basato su classi e relazioni tra oggetti.

Le principali entità del sistema sono:

- **SistemaSolare**: questa classe funge da contenitore principale dei dati relativi ai pianeti. Contiene una lista di oggetti di tipo Pianeta e l'immagine del Sole che andrà messo al centro della pagina.
- **Pianeta**: rappresenta un singolo pianeta visualizzato nel sistema caratterizzandolo con il nome, la descrizione, l'immagine e un link al video informativo. Queste informazioni andranno a popolare la pagina che descriverà ogni singolo pianeta.
- **Luna**: questa classe si occuperà di mostrare l'immagine della luna e alle sue relative fasi lunari. Viene associata al Sistema Solare pur non facendone parte perché si trova nella stessa pagina.
- **APINASA**: la seguente classe si occuperà di fornire le informazioni, prese direttamente dal sito della NASA, alla pagina che si occupa della curiosità del giorno.
- **CuriositàDelGiorno**: essa mostra le informazioni prese dal API, che ogni giorno vengono aggiornate.

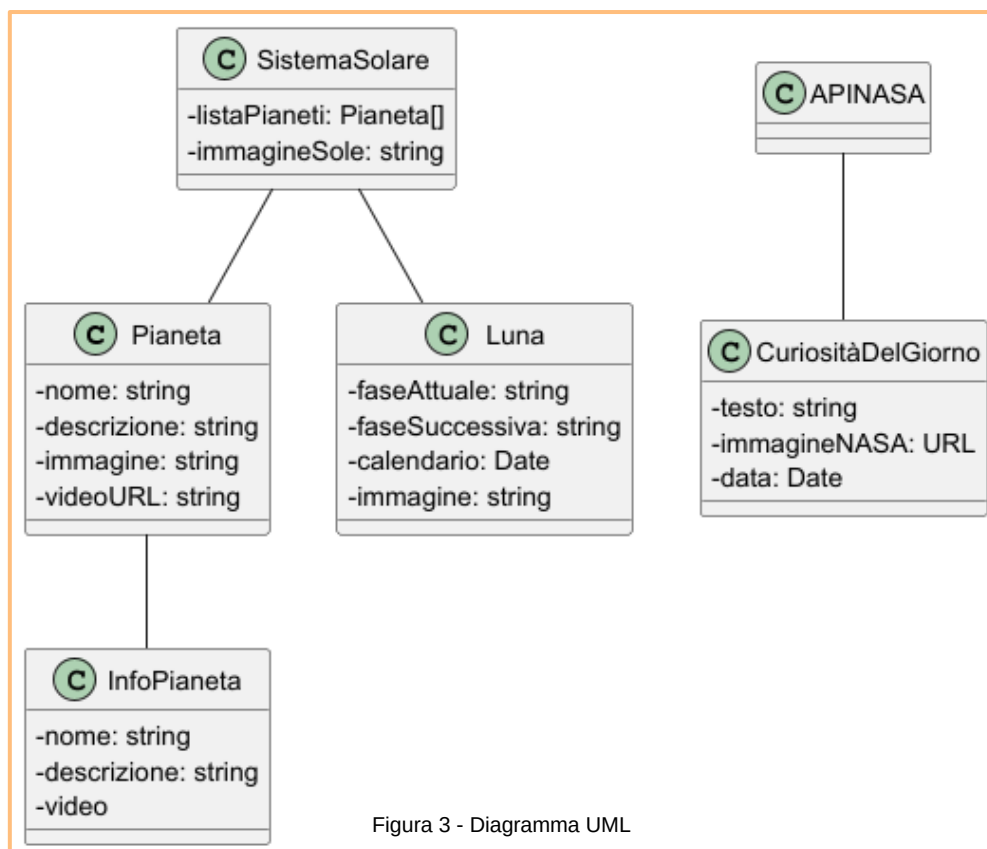


Figura 3 - Diagramma UML

5.3 Design delle interfacce

La progettazione delle interfacce definisce l'aspetto grafico e le modalità con cui l'utente interagisce con il sistema.

Le interfacce sono state progettate sulla base dei requisiti raccolti nella fase di analisi (funzionali e non funzionali) e sono state rappresentate tramite Mockup che guidano la fase di sviluppo.

Il design segue uno stile coerente con il tema astronomico, privilegiando leggibilità, semplicità e intuitività nell'interazione.

Tutte le interfacce sono state pensate per l'utilizzo su Apple, in particolare su un Ipad.

5.3.1 Pagina principale

Questa è l'interfaccia principale, è molto semplice così da renderla comprensibile anche agli utenti che hanno difficoltà nel comprendere l'uso degli applicativi.

Essa mostra una scritta che dà il benvenuto all'utente e un bottone che una volta premuto porterà alla pagina successiva.

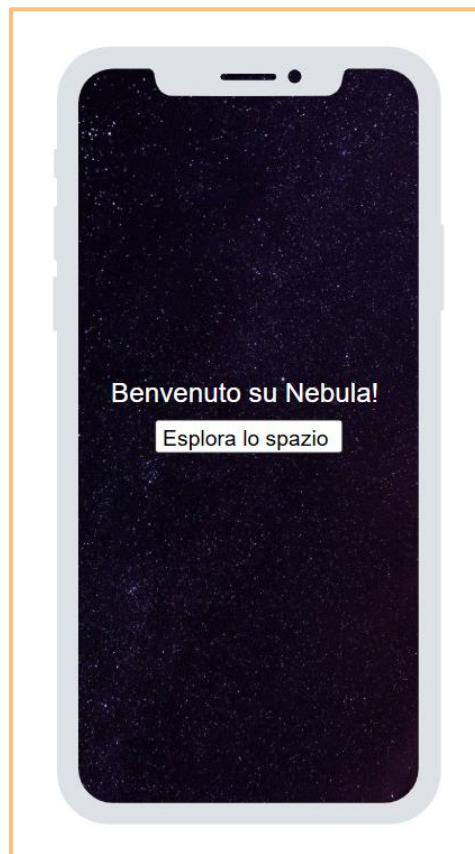


Figura 4 - Pagina principale Mockup

5.3.2 Sistema Solare

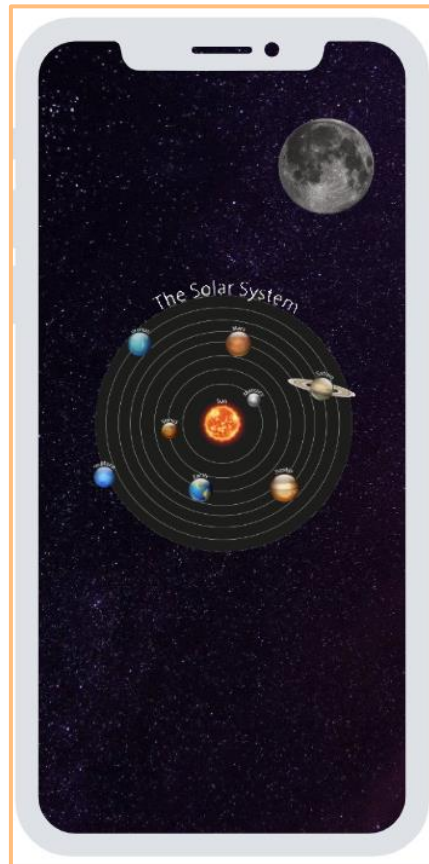


Figura 5 - Sistema Solare Mockup

Dopo aver premuto il pulsante presente nella schermata iniziale, l'utente accede a una seconda interfaccia interattiva.

In questa schermata viene visualizzato il Sistema Solare: al centro è posizionato il Sole e attorno ad esso sono disposte delle orbite, lungo le quali i pianeti ruoteranno in maniera animata.

Sebbene non sia un pianeta, è presente anche la Luna in quanto satellite naturale della Terra. Nel modello progettato, la Luna è quindi rappresentata come elemento aggiuntivo.

5.3.3 Informazioni sui pianeti



Figura 6 - Informazioni sui pianeti Mockup

La pagina dedicata alle informazioni dei singoli pianeti sarà accessibile quando l'utente selezionerà un pianeta dall'interfaccia del Sistema Solare.

Ogni pianeta avrà una schermata informativa dedicata che conterrà un'immagine rappresentativa, una descrizione testuale e un video esplicativo proveniente dal canale YouTube della National Geographic, al fine di fornire contenuti scientificamente accurati e di supporto visivo.

5.3.4 Informazioni sulla Luna

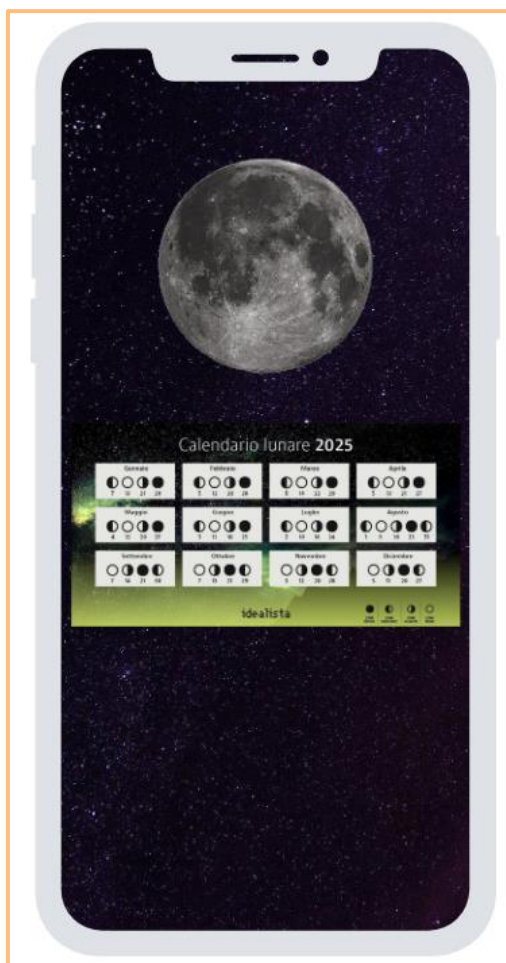


Figura 7 - Informazioni Luna Mockup

Come per i pianeti, anche la Luna avrà una pagina dedicata. Quando l'utente selezionerà la Luna, verrà aperta una schermata che mostrerà un'immagine e il calendario lunare con la visualizzazione delle varie fasi (nuova, crescente, piena, calante).

5.3.5 L'immagine del giorno



Figura 8 - APOD Mockup

All'interno della schermata del Sistema Solare sarà presente un pulsante dedicato che consentirà l'accesso all'ultima sezione dell'applicazione: la pagina della *"Foto del Giorno"*. In questa schermata verrà mostrata un'immagine astronomica accompagnata da una breve descrizione. Il contenuto verrà aggiornato quotidianamente grazie all'integrazione con l'API esterna, la quale fornirà automaticamente una nuova immagine ogni giorno.

5.3.6 Galassia

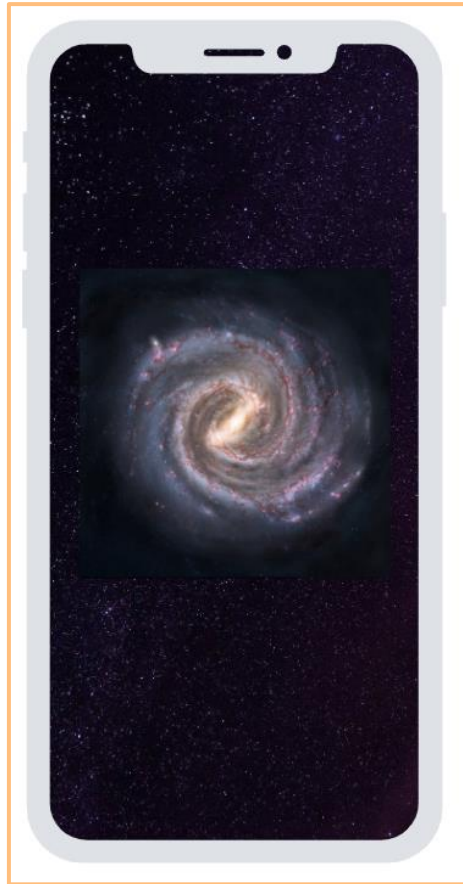


Figura 9 - Galassia Mockup

L'applicazione permetterà all'utente di ingrandire o ridurre la visualizzazione della schermata del Sistema Solare tramite controlli dedicati.

Quando la schermata verrà ridotta, la visualizzazione passerà da una prospettiva centrata sul Sistema Solare a una vista più ampia che mostrerà la galassia di appartenenza, ovvero la *Via Lattea*.



5.4 Design procedurale

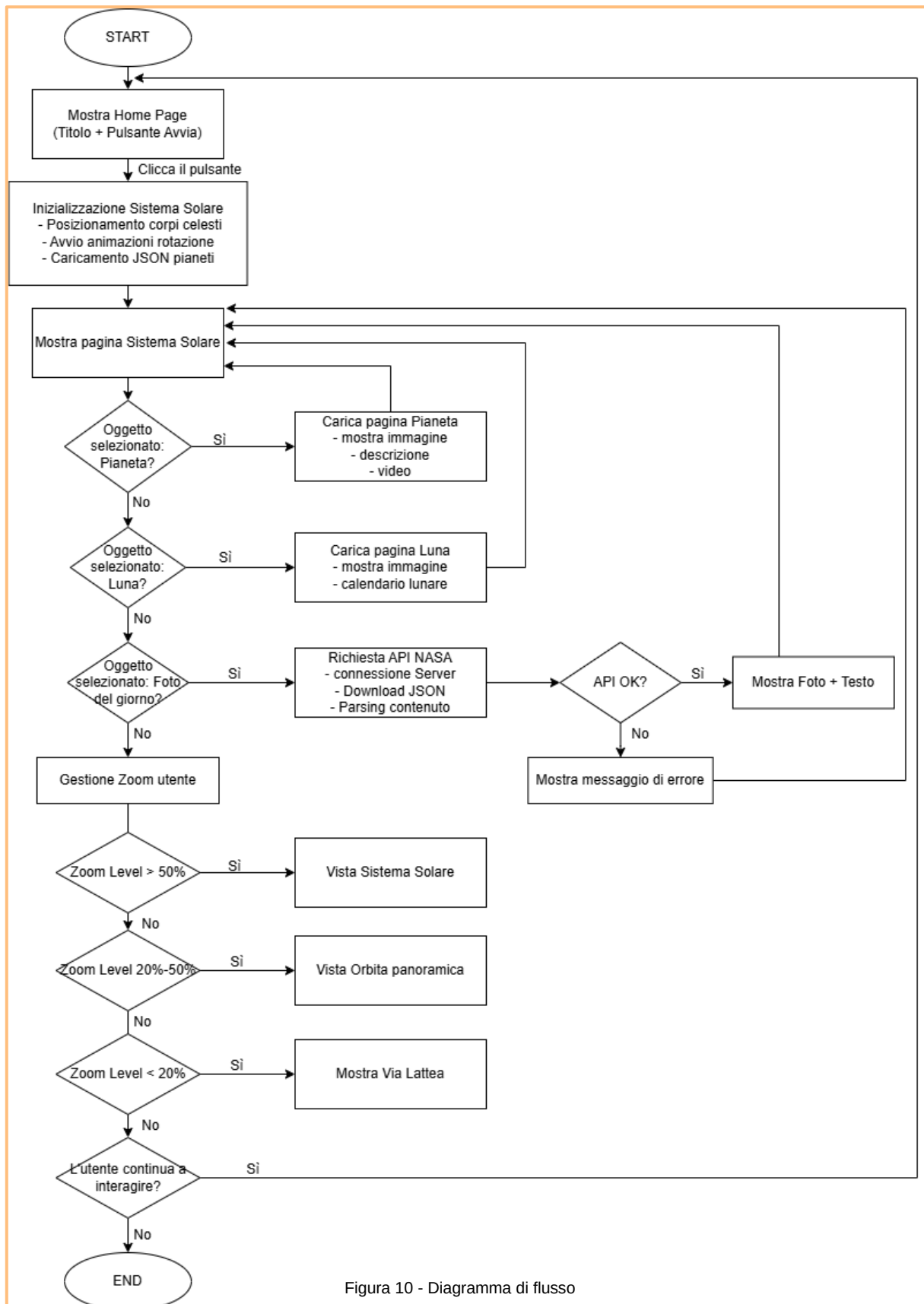


Figura 10 - Diagramma di flusso

	SAMT – Sezione Informatica	Pagina 23 di 45
	Implementazione di Nebula in Swift	

Il diagramma di flusso definisce il comportamento dinamico dell'applicazione e descrive in modo sistematico le azioni che l'utente può compiere e le corrispondenti risposte del sistema.

L'applicazione inizia con la fase di inizializzazione dell'interfaccia. Dopo l'accesso alla schermata principale, l'utente può interagire con il modello animato del Sistema Solare e accedere a contenuti informativi o multimediali selezionando i pianeti o la Luna.

Una parte specifica del flusso gestisce la funzionalità "Foto del Giorno", la quale utilizza un servizio API esterno della NASA. Tale blocco include anche un percorso alternativo in caso di errore di connessione o risposta non valida, garantendo stabilità e continuità dell'esperienza utente.

Il sistema include inoltre una logica di zoom dinamico che modifica progressivamente la prospettiva visuale fino a mostrare la Via Lattea quando il livello di zoom raggiunge valori minimi.

Il flusso non presenta una conclusione lineare, ma un ciclo operativo illimitato che permette all'utente di continuare a navigare tra le schermate finché decide di lasciare l'applicazione.



6 Implementazione

6.1 Creazione dell'applicativo

La fase di implementazione rappresenta il passaggio dalla progettazione astratta alla realizzazione concreta dell'applicazione.

In questa fase sono state sviluppate tutte le componenti software previste dall'analisi dei requisiti e dal design architetturale, traducendo i modelli concettuali e i diagrammi realizzati nelle fasi precedenti in codice eseguibile.

L'applicazione è stata implementata utilizzando il linguaggio Swift all'interno dell'ambiente Swift Playgrounds, scelti per la loro compatibilità con animazioni fluide, interazioni touch e gestione multimediale, elementi fondamentali per un progetto didattico-interattivo come quello realizzato.

L'architettura software è stata strutturata in modo modulare e organizzata in otto classi principali, ognuna con una responsabilità ben definita, al fine di garantire scalabilità, manutenibilità e chiarezza del codice.

Questa organizzazione ha consentito di suddividere le funzionalità in componenti indipendenti ma comunicanti tra loro, permettendo un'implementazione ordinata e coerente con i principi della programmazione a oggetti.

6.2 ContentView.swift

ContentView è la vista iniziale dell'applicazione. Fornisce un'immagine di sfondo a schermo intero e un'interfaccia molto semplice composta da un titolo e un pulsante che, quando premuto, esegue la navigazione verso HomeView tramite un *NavigationStack*. L'immagine di sfondo è resa con *.scaledToFill()* e *ignoresSafeArea()* per coprire l'intero schermo; il pulsante è implementato come *NavigationLink* stilizzato (sfondo bianco, testo nero, angoli arrotondati e ombra). Per garantire compatibilità e accessibilità si raccomanda di evitare dimensioni fisse per il layout, di aggiungere etichette di accessibilità e di localizzare tutte le stringhe.

6.2.1 Codice significativo

```
NavigationLink(destination: HomeView()) {
    Text("Explore space, one planet at a time")
        .foregroundColor(.black)
        .font(.title2)
        .frame(width: 450, height: 50)
        .background(Color.white)
        .cornerRadius(25)
        .shadow(radius: 5)
}
```

Figura 11 - NavigationLink

Il pulsante principale è implementato attraverso un *NavigationLink* che conduce alla schermata successiva (HomeView). L'elemento è stilizzato con bordi arrotondati, ombra e dimensioni personalizzate per renderlo ben visibile e facilmente selezionabile dall'utente.

6.2.2 Interfaccia

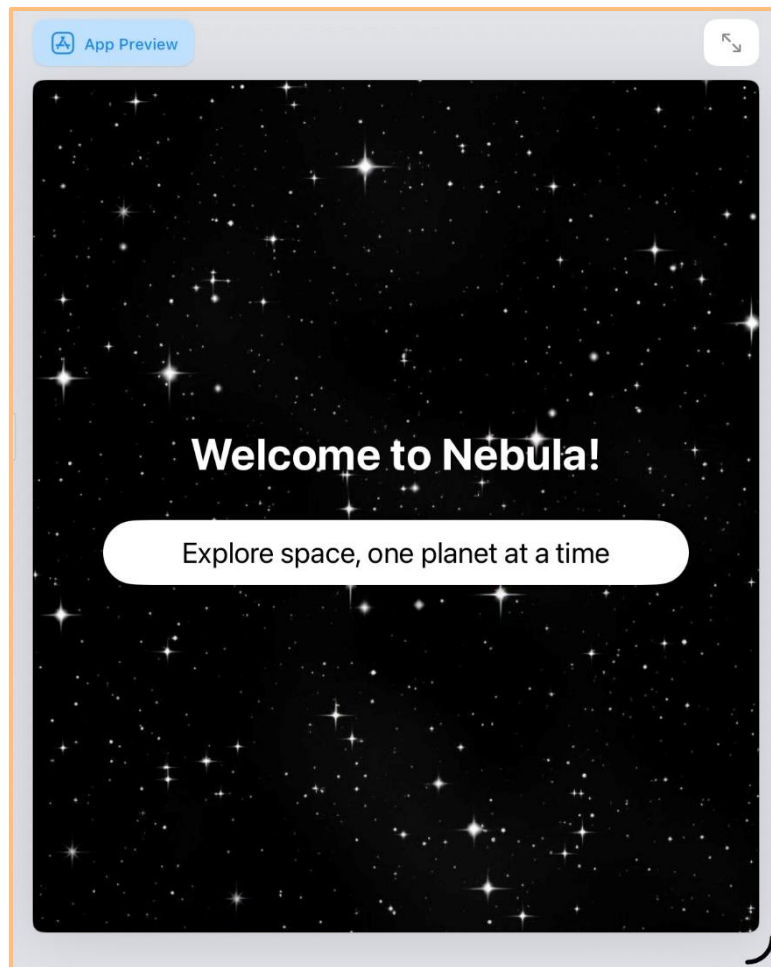


Figura 12 - Pagina iniziale

La ContentView ha permesso di creare la pagina iniziale; grazie ai suoi metodi e alla struttura modulare di SwiftUI, è stato possibile definire l'aspetto e il comportamento dell'interfaccia in modo chiaro e personalizzato. Questa schermata rappresenta il punto di ingresso dell'applicazione e svolge un ruolo fondamentale nell'esperienza utente, poiché introduce la navigazione verso le diverse sezioni disponibili.



6.3 HomeView.swift

HomeView rappresenta la pagina principale dell'applicazione e permette all'utente di interagire con una rappresentazione simulata del Sistema Solare. La schermata mostra il Sole al centro e i pianeti disposti su orbite animate. Ogni pianeta può essere selezionato tramite un *NavigationLink*, che conduce alla pagina informativa dedicata. L'interfaccia supporta anche un gesto di zoom: se il valore di ingrandimento scende sotto una soglia predefinita, la rappresentazione del Sistema Solare si trasforma in una vista più ampia della Via Lattea. La schermata include inoltre un accesso diretto alla pagina della Luna e una toolbar che consente la navigazione verso la funzionalità "Information of the day" collegata a un'API NASA. La struttura della view combina componenti statici (immagini e layout) con elementi dinamici e interattivi, rendendo la pagina sia esplorativa che informativa.

6.3.1 Codice significativo

```
ForEach(planetsData, id: \.name) { planet in
    if let orbital = orbitals.first(where: { $0.0 == planet.name }) {
        NavigationLink(destination: PlanetDetailView(planet: planet)) {
            PlanetView(imageName: planet.name,
                        radius: orbital.1,
                        duration: orbital.2,
                        size: orbital.3)
        }
        .buttonStyle(PlainButtonStyle())
    }
}
```

Figura 13 – Accesso a PlanetDetailView

Questo blocco di codice crea una lista dinamica di pianeti basata sui dati forniti. Ogni elemento viene verificato con un array di informazioni orbitali e, se disponibile, viene mostrato all'interno di un *NavigationLink*. Toccando un pianeta, l'utente accede alla schermata di dettaglio corrispondente. La vista del pianeta viene reindirizzata tramite *PlanetView*, alla quale vengono passati i valori orbitali estratti dall'array *orbitals*. Il bottone utilizza *PlainButtonStyle()* per rimuovere lo stile standard e mantenere l'aspetto personalizzato.



6.3.2 Interfaccia

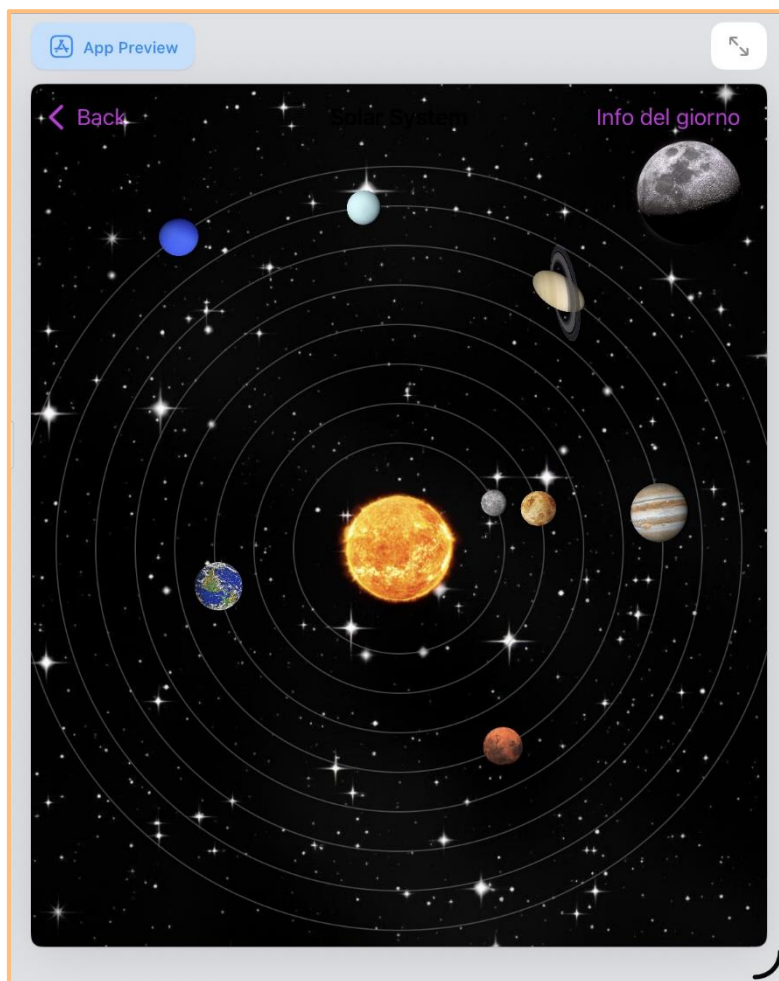


Figura 14 - Sistema Solare

Questa pagina rappresenta un modello grafico del Sistema Solare realizzato all'interno dell'app. Al centro è posizionato il Sole, circondato da una serie di orbite che indicano i percorsi dei pianeti. Su ciascuna orbita è collocata un'immagine rappresentativa di un pianeta, disposta in modo da riprodurre la configurazione del Sistema Solare. Lo sfondo è composto da un cielo stellato, che contribuisce a creare un'atmosfera spaziale immersiva.

Nella parte superiore sono presenti due elementi dell'interfaccia:

- “Back” (a sinistra), che permette di tornare alla schermata precedente.
- “Info del giorno” (a destra), pulsante che consente di accedere a una sezione informativa aggiuntiva.



6.4 InfoView.swift

InfoView è la schermata dedicata alla funzione "Info del giorno". Essa utilizza un modello architetturale basato su MVVM, dove APODViewModel gestisce la richiesta della risorsa APOD¹ fornita dalla NASA tramite API REST. La view effettua la chiamata automaticamente all'apparizione della schermata (*onAppear*) e mostra dinamicamente il contenuto ricevuto, includendo immagine, titolo e descrizione. La UI è progettata per gestire correttamente stati di caricamento, errori di rete e visualizzazione remota di immagini tramite *AsyncImage*. Questa funzionalità arricchisce l'app con contenuti aggiornati e scientificamente verificati, migliorando l'esperienza didattica e interattiva dell'utente.

6.4.1 Codice significativo

```
class APODViewModel: ObservableObject {
    @Published var apod: APOD?
    @Published var errorMessage: String?

    private let apiKey = "7mcnzjS2pAluJZlNUjGxOY74dIX3fGRKsklpkQqm"

    func fetchAPOD() {
        let urlString = "https://api.nasa.gov/planetary/apod?api_key=\(apiKey)"
        guard let url = URL(string: urlString) else {
            self.errorMessage = "URL non valido"
            return
        }
        let request = URLRequest(url: url)
        URLSession.shared.dataTask(with: request) { data, response, error in
            if let error = error {
                DispatchQueue.main.async {
                    self.errorMessage = "Error: \(error.localizedDescription)"
                }
                return
            }
            guard let data = data else {
                DispatchQueue.main.async {
                    self.errorMessage = "Nessun dato ricevuto"
                }
                return
            }
            do {
                let decoded = try JSONDecoder().decode(APOD.self, from: data)
                DispatchQueue.main.async {
                    self.apod = decoded
                }
            } catch {
                DispatchQueue.main.async {
                    self.errorMessage = "Errore di decodifica: \(error)"
                }
            }
        }.resume()
    }
}
```

Figura 15 - API

Questo ViewModel gestisce il processo di comunicazione con l'API NASA APOD, occupandosi di scaricare i dati relativi alla foto astronomica del giorno. Quando viene invocato *fetchAPOD()*, viene creata e inviata una richiesta HTTP verso l'endpoint ufficiale dell'API. La risposta ricevuta viene analizzata: se il server restituisce dati validi, questi vengono decodificati dal formato JSON nel modello Swift APOD.

¹ Astronomy Picture of the Day

Durante questo processo il ViewModel gestisce anche eventuali errori, come problemi di rete, risposta non valida o errori di decodifica, e li rende disponibili attraverso una proprietà osservabile dedicata (*errorMessage*). Sia il dato ottenuto che gli errori vengono pubblicati tramite proprietà *@Published*, garantendo che qualsiasi vista collegata possa aggiornarsi automaticamente.

Tutte le modifiche allo stato osservabile avvengono sul *main thread*, assicurando compatibilità con SwiftUI e impedendo comportamenti imprevisti nell'interfaccia. In questo modo la UI può reagire in modo fluido e coerente alla disponibilità dei dati o ad eventuali problemi durante la richiesta.

6.4.2 Interfaccia

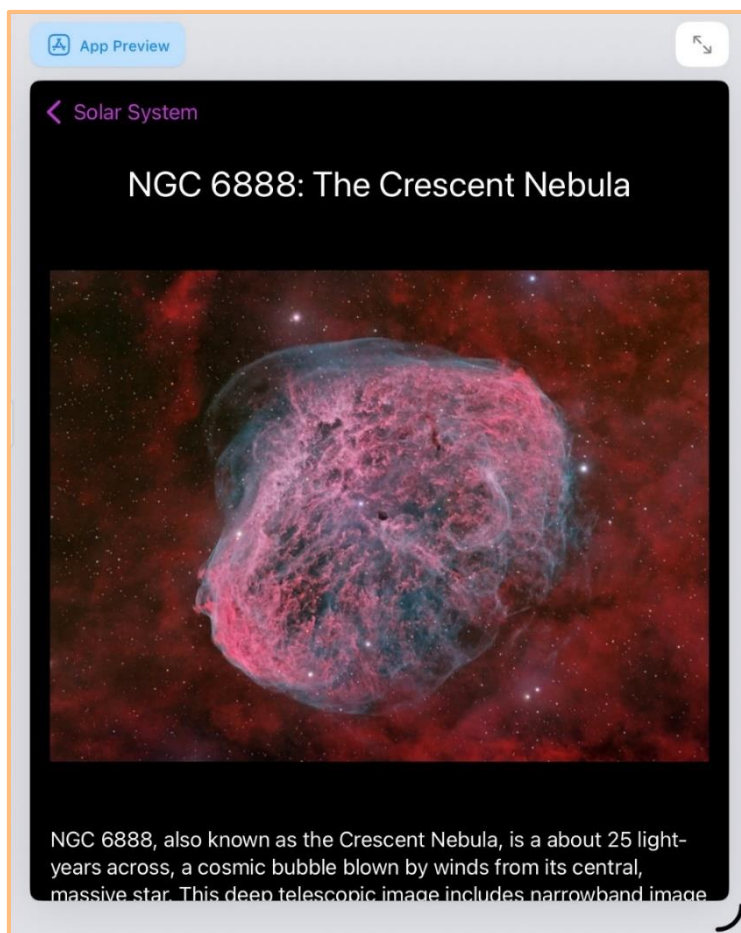


Figura 16 - Interfaccia APOD

Questa schermata mostra il risultato di una richiesta a una API astronomica utilizzata dall'app per ottenere ogni giorno un contenuto scientifico aggiornato. In questo caso, l'API ha restituito le informazioni relative alla NGC 6888 – Crescent Nebula, un oggetto celeste visivamente molto riconoscibile.

Nella parte superiore è presente il pulsante Back, che consente di tornare alla pagina principale. Subito sotto compare il titolo dinamico dell'oggetto astronomico, recuperato direttamente dalla risposta dell'API.

Al centro della schermata viene visualizzata l'immagine ad alta risoluzione fornita dalla stessa API. L'immagine viene caricata e adattata automaticamente al layout dell'app.

Nella parte inferiore è presente una descrizione testuale, anch'essa generata a partire dai dati dell'API. Il testo include informazioni scientifiche come dimensioni, distanza e caratteristiche fisiche.



6.5 MyApp.swift

MyApp rappresenta il punto di ingresso dell'applicazione. Utilizza l'attributo `@main` per indicare la struttura principale dell'app, conforme al protocollo App, che definisce il ciclo di vita dell'applicazione in ambiente SwiftUI. All'interno della proprietà `body`, viene dichiarato un `WindowGroup` che crea la finestra principale dell'app contenente la schermata iniziale `ContentView()`. Da questa struttura inizia l'esecuzione dell'intera interfaccia e la navigazione verso le altre viste. Il file funge quindi da configurazione base dell'app e garantisce l'avvio corretto dell'esperienza utente.

6.6 Planet.swift

Il file **Planet** definisce le strutture dati utilizzate per rappresentare i pianeti e i contenuti multimediali associati. Le strutture Planet e Video adottano il protocollo `Codable`, permettendo la lettura e la decodifica dei dati da un file JSON. Ogni pianeta possiede un nome, una lista di dettagli informativi e, se disponibile, un video esplicativo proveniente da YouTube tramite il modello Video. Questo approccio consente una gestione dinamica e strutturata delle informazioni, rendendo il sistema facilmente espandibile.

6.6.1 Codice significativo

```
import Foundation

struct Video: Codable {
    let url: String
    let title: String?
}

struct Planet: Codable {
    let name: String
    let details: [String]
    let video: Video?
}
```

Figura 17 – Struttura JSON

Nel progetto sono stati definiti due tipi di dati che servono per organizzare meglio le informazioni sui pianeti e sui loro contenuti multimediali:

- Video: rappresenta un singolo video collegato a un pianeta. Contiene due informazioni:
 - l'indirizzo del video: il video è stato salvato in formato mp4 e salvato nella cartella del progetto. Per convertire il video YouTube in formato mp4 è stato utilizzato [TurboScribe](#).
 - il titolo del video, che è opzionale.

Serve quindi per associare a un pianeta un contenuto video da mostrare all'utente.

- Planet: rappresenta un pianeta singolo. Contiene:
 1. il nome del pianeta;
 2. una lista di dettagli o descrizioni;
 3. un eventuale video, che usa il tipo Video descritto precedentemente.

Questo permette di gestire tutte le informazioni presenti in ognuna delle pagine dedicate ai pianeti, avendo la stessa struttura.



6.7 PlanetData.swift

Il file **PlanetData** implementa una classe responsabile del caricamento dei dati relativi ai pianeti da un file JSON incluso nel progetto. Il metodo statico *load()* individua il file nel bundle dell'applicazione, ne legge il contenuto e lo decodifica in una lista di oggetti Planet tramite *JSONDecoder*. In caso di errore (file mancante o struttura JSON non valida), il metodo restituisce un array vuoto e stampa un messaggio diagnostico. Questo meccanismo consente di separare i dati dalla logica applicativa, garantendo scalabilità, modularità e semplicità nella gestione e nell'aggiornamento dei contenuti.

6.7.1 Codice significativo

```
class PlanetData {
    static func load() -> [Planet] {
        guard let url = Bundle.main.url(forResource: "json", withExtension: "json") else {
            print("Errore: file json.json non trovato")
            return []
        }
        do {
            let data = try Data(contentsOf: url)
            let decoded = try JSONDecoder().decode([Planet].self, from: data)
            return decoded
        } catch {
            print("Errore nel caricamento JSON: \(error)")
            return []
        }
    }
}
```

Figura 18 - Gestione errori

Il metodo *load()* si occupa di caricare il file JSON e convertirlo in un array di oggetti Planet. La funzione esegue i seguenti passaggi:

- Ricerca del file JSON: utilizza *Bundle.main.url* per individuare il file *json.json*. Se il file non è presente, restituisce un array vuoto e registra un messaggio di errore;
- Lettura del contenuto del file: una volta trovato il file, tenta di leggerne il contenuto tramite *Data(contentsOf:)*;
- Decodifica del JSON in oggetti Swift: il contenuto viene decodificato con *JSONDecoder*, convertendolo in un array di Planet. In caso di successo, i dati vengono restituiti;
- Gestione degli errori: attraverso un blocco *do/catch* vengono gestiti gli errori.

Questa classe è fondamentale poiché permette di riconoscere gli errori, assicura la scalabilità e facilita eventuali aggiornamenti.



6.8 PlanetDetailView.swift

Il file **PlanetDetailView** implementa l'interfaccia dedicata alla visualizzazione dei contenuti informativi relativi ai pianeti.

La vista riceve in entrata un'istanza del modello Planet e costruisce dinamicamente il layout mostrando nome, immagine associata, descrizioni testuali e un video esplicativo riprodotto tramite AVPlayer.

La disposizione degli elementi è gestita tramite una *ScrollView* per permettere la consultazione completa dei contenuti anche su schermi di dimensioni ridotte. L'interfaccia mantiene coerenza estetica con il resto dell'applicazione utilizzando sfondo tematico e palette cromatica appropriata.

Questa struttura modulare consente di aggiornare o estendere le informazioni dei pianeti modificando esclusivamente il dataset JSON, senza richiedere interventi sulla logica implementata.

6.8.1 Codice significativo

```
import AVKit

struct VideoButton: View {
    let localVideoURL: URL
    let title: String
    @State private var showPlayer = false

    var body: some View {
        VStack(alignment: .leading, spacing: 10) {
            Button(action: {
                showPlayer = true
            }) {
                Label(title, systemImage: "play.circle.fill")
                    .foregroundColor(.white)
                    .font(.headline)
            }
            .sheet(isPresented: $showPlayer) {
                VideoPlayer(player: AVPlayer(url: localVideoURL))
                    .edgesIgnoringSafeArea(.all)
            }
        }
    }
}
```

Figura 19 – Gestione dei video

Il componente VideoButton è una vista SwiftUI progettata per presentare un pulsante che consente all'utente di avviare la riproduzione di un video locale.

La vista accetta due parametri:

- *localVideoURL*, un oggetto URL che identifica il file video da riprodurre;
- *title*, una stringa utilizzata come etichetta descrittiva del pulsante.

La gestione dello stato è affidata alla proprietà *@State showPlayer*, che controlla la visualizzazione della sheet modale contenente il lettore video.

Al momento dell'interazione dell'utente con il pulsante, la variabile *showPlayer* viene impostata a true, determinando l'apertura della sheet.

All'interno della sheet viene istanziato un VideoPlayer basato su AVPlayer, configurato per riprodurre il contenuto video puntato dall'URL fornito.

La modifica *.edgesIgnoringSafeArea(.all)* consente al lettore di estendersi sull'intera area dello schermo, ignorando i margini della safe area e garantendo un'esperienza di visualizzazione a schermo pieno.

6.8.2 Interfaccia

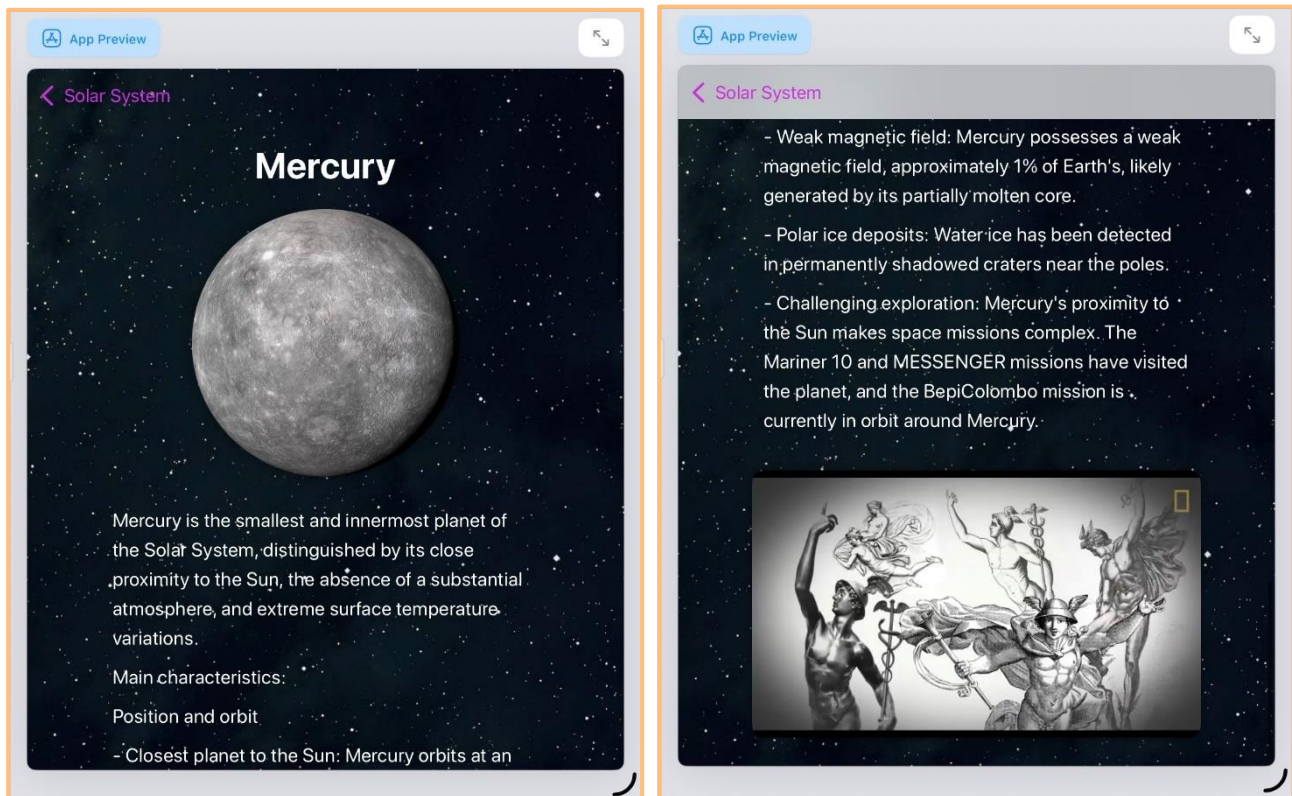


Figura 20 - Informazioni sul pianeta

Le pagine dedicate ai pianeti hanno la stessa struttura, ovvero: nome del pianeta, immagine, descrizione e video informativo. Questo permette ad un utente meno esperto di comprendere facilmente la pagina e concentrarsi sulle informazioni fornite.



6.9 PlanetView.swift

Il file **PlanetView** definisce una vista riutilizzabile che simula graficamente il movimento orbitale dei pianeti. Ogni istanza della vista genera un'orbita tramite un cerchio e posiziona al suo interno l'immagine del pianeta, che viene successivamente animata attraverso una rotazione continua attorno al centro della vista. I parametri configurabili permettono di adattare dinamicamente dimensione, raggio orbitale e velocità, rendendo il componente flessibile e scalabile. L'animazione, avviata automaticamente al caricamento della vista, contribuisce a una rappresentazione visivamente coinvolgente del Sistema Solare.

6.9.1 Codice significativo

```
@State private var angle: Double = 0

var body: some View {
    ZStack {
        Circle()
            .stroke(Color.gray.opacity(0.5), lineWidth: 1)
            .frame(width: radius * 2, height: radius * 2)

        Image(imageName)
            .resizable()
            .frame(width: size, height: size)
            .offset(x: radius)
            .rotationEffect(.degrees(angle))
            .onAppear {
                withAnimation(.linear(duration: duration).repeatForever(autoreverses: false)) {
                    angle = 360
                }
            }
    }
}
```

Figura 21 - Rotazione

Questo componente SwiftUI definisce una vista che rappresenta un'immagine in rotazione attorno a un cerchio tracciato come guida visiva.

La proprietà `@State private var angle` viene utilizzata per controllare l'angolo di rotazione dell'immagine e per animarne il movimento continuo.

La vista è composta da uno `ZStack`, all'interno del quale:

- Cerchio di riferimento: viene disegnato un cerchio tramite `Circle().stroke(...)`, utilizzato come percorso visivo su cui l'immagine effettua la rotazione. Il cerchio è dimensionato mediante `frame(width: radius * 2, height: radius * 2)` per riflettere il raggio specificato dal chiamante.
- Immagine rotante: l'immagine identificata dal parametro `imageName` viene resa ridimensionabile e adattata alle dimensioni fornite. Successivamente viene posizionata in offset sull'asse X in modo da collocarsi lungo la circonferenza del cerchio (`offset(x: radius)`), quindi viene applicata una rotazione proporzionale al valore della variabile `angle`.
- Animazione continua: all'interno del modificatore `.onAppear`, viene impostata un'animazione lineare (`withAnimation(.linear(...))`) che incrementa il valore di `angle` fino a 360°. Grazie a `.repeatForever(autoreverses: false)`, la rotazione viene ripetuta indefinitamente senza inversioni, producendo un movimento circolare costante.

7 Test

7.1 Protocollo di test

Il seguente capitolo si occupa dei test, coloro che permettono di verificare il corretto funzionamento del programma. Ogni azione che l'utente andrà a fare bisognerà testarlo.

Test Case:	TC-001	Nome:	Avvio dell'applicativo
Riferimento:	REQ-01		
Descrizione:	Verificare il corretto avvio dell'applicativo.		
Prerequisiti:	È necessario avere tutta la cartella che contiene tutti i file dell'applicativo.		
Procedura:	1. Scaricare Swift Playground; 2. Importare la cartella del Progetto; 3. Avviare l'applicativo.		
Risultati attesi:	Riuscire a vedere la pagina iniziale.		

Test Case:	TC-002	Nome:	Accedere alla pagina del Sistema Solare
Riferimento:	REQ-01		
Descrizione:	Il bottone presente nella pagina iniziale permette di passare alla pagina dedicata al Sistema Solare.		
Prerequisiti:	È necessario aver verificato il TC-001.		
Procedura:	1. Avviare l'applicativo; 2. Premere il bottone.		
Risultati attesi:	Riuscire a passare alla pagina successiva.		

Test Case:	TC-003	Nome:	Sistema Solare
Riferimento:	REQ-02		
Descrizione:	La seconda pagina mostra il Sole al centro e i rispettivi pianeti che ruotano attorno ad esso. Inoltre, si vedrà la Luna e una barra di navigazione che permette di tornare alla pagina precedente (Back) oppure passare a quella successiva (Picture of the Day).		
Prerequisiti:	È necessario aver verificato il TC-002.		
Procedura:	1. Avviare l'applicativo; 2. Premere il bottone della pagina iniziale.		
Risultati attesi:	Riuscire a vedere la pagina del Sistema Solare con i pianeti che orbitano attorno al Sole, la Luna e la barra di navigazione.		

Test Case:	TC-004	Nome:	Ingrandimento della pagina sul Sistema Solare
Riferimento:	REQ-03		
Descrizione:	La pagina dedicata al Sistema Solare sarà possibile ingrandirla, in modo tale da poter vedere i dettagli.		
Prerequisiti:	È necessario aver verificato il TC-003.		
Procedura:	1. Avviare l'applicativo; 2. Premere il bottone della pagina iniziale;		

	3. Ingrandire la pagina.
Risultati attesi:	Riuscire a vedere la pagina del Sistema Solare da più vicino.

Test Case:	TC-005	Nome:	Rimpicciolimento della pagina sul Sistema Solare
Riferimento:	REQ-04		
Descrizione:	La pagina dedicata al Sistema Solare sarà possibile rimpicciolirla, in modo tale da poter vedere la Via Lattea.		
Prerequisiti:	È necessario aver verificato il TC-003.		
Procedura:	1. Avviare l'applicativo; 2. Premere il bottone della pagina iniziale; 3. Rimpicciolire la pagina.		
Risultati attesi:	Riuscire a vedere la Via Lattea.		

Test Case:	TC-006	Nome:	Scelta del pianeta
Riferimento:	REQ-05		
Descrizione:	Si potrà premere uno dei pianeti a scelta, in seguito si passerà ad una pagina che riprenderà le informazioni più importanti del pianeta scelto.		
Prerequisiti:	È necessario aver verificato il TC-003.		
Procedura:	1. Avviare l'applicativo; 2. Premere il bottone della pagina iniziale; 3. Premere uno dei pianeti a scelta.		
Risultati attesi:	Riuscire a vedere la pagina dedicata con tutte le informazioni.		

Test Case:	TC-007	Nome:	Informazioni sul pianeta
Riferimento:	REQ-06		
Descrizione:	Premendo uno dei pianeti si passerà ad una pagina successiva, ovvero a quella che contiene tutte le informazioni sul pianeta scelto.		
Prerequisiti:	È necessario aver verificato il TC-006.		
Procedura:	1. Avviare l'applicativo; 2. Premere il bottone della pagina iniziale; 3. Premere uno dei pianeti a scelta; 4. Scorrere la pagina delle informazioni.		
Risultati attesi:	Riuscire a vedere le informazioni sul rispettivo pianeta.		

Test Case:	TC-008	Nome:	Video dedicato al pianeta
Riferimento:	REQ-06		
Descrizione:	All'interno della pagina che si occupa delle informazioni sui pianeti, sarà presente un video didattico sul pianeta della National Geographic.		
Prerequisiti:	È necessario aver verificato il TC-006. È necessario aver esportato tutta la cartella dell'applicativo la quale contiene i video in formato .mp4.		
Procedura:	1. Avviare l'applicativo;		

	2. Premere il bottone della pagina iniziale; 3. Premere uno dei pianeti a scelta; 4. Scorrere la pagina delle informazioni; 5. Avviare il video.
Risultati attesi:	Riuscire a vedere e sentire il video.

Test Case:	TC-009	Nome:	Premere la Luna
Riferimento:	REQ-07		
Descrizione:	Premendo la Luna si potrà passare ad una pagina successiva.		
Prerequisiti:	È necessario aver verificato il TC-003.		
Procedura:	1. Avviare l'applicativo; 2. Premere il bottone della pagina iniziale; 3. Premere la Luna.		
Risultati attesi:	Riuscire a passare ad un'ulteriore pagina.		

Test Case:	TC-010	Nome:	Informazioni sulla Luna
Riferimento:	REQ-07		
Descrizione:	Premendo la Luna si potrà passare ad una pagina dedicata ad essa, in cui viene mostrata la fase attuale e il calendario lunare dell'anno 2025.		
Prerequisiti:	È necessario aver verificato il TC-009. È necessario avere la connessione ad Internet per poter vedere la fase attuale.		
Procedura:	1. Avviare l'applicativo; 2. Premere il bottone della pagina iniziale; 3. Premere la Luna; 4. Scorrere la pagina delle informazioni.		
Risultati attesi:	Riuscire a vedere la fase attuale in tempo reale e il calendario lunare.		

Test Case:	TC-011	Nome:	Informazione del giorno
Riferimento:	REQ-08		
Descrizione:	Premendo il riferimento "Picture of the Day" si passerà ad una pagina che si occupa, tramite un API NASA, di fornire la foto scattata dalla NASA in quel giorno con una breve descrizione.		
Prerequisiti:	È necessario aver verificato il TC-003. È necessario avere la connessione ad Internet per la seguente pagina.		
Procedura:	1. Avviare l'applicativo; 2. Premere il bottone della pagina iniziale; 3. Premere il riferimento "Picture of the Day"; 4. Scorrere la pagina delle informazioni.		
Risultati attesi:	Riuscire a vedere la foto del giorno e la rispettiva descrizione.		

Test Case:	TC-012	Nome:	Barra di navigazione
	REQ-01		

Riferimento:	REQ-08	
Descrizione:	La barra di navigazione deve permettere di passare da una pagina all'altra senza problemi.	
Prerequisiti:	È necessario aver verificato il TC-001.	
Procedura:	1. Avviare l'applicativo.	
Risultati attesi:	Riuscire a passare da una pagina all'altra.	

7.2 Risultati test

Test	Data	Risultato
TC-01	12.12.2025	Passato
TC-02	12.12.2025	Passato
TC-03	12.12.2025	Passato
TC-04	12.12.2025	Passato
TC-05	12.12.2025	Passato
TC-06	12.12.2025	Passato
TC-07	12.12.2025	Passato
TC-08	12.12.2025	Passato
TC-09	12.12.2025	Passato
TC-10	12.12.2025	Passato
TC-11	12.12.2025	Passato
TC-12	12.12.2025	Passato

Figura 22 - Risultato dei test

7.3 Mancanze/limitazioni conosciute

Nel progetto sono presenti alcune limitazioni dovute sia a scelte di semplificazione sia ai vincoli dell'ambiente di sviluppo. La funzione di zoom risulta piuttosto basilare, poiché permette solo un ingrandimento e un rimpicciolimento semplice, senza offrire un controllo più preciso della scena o la possibilità di ruotare la visualizzazione. Anche le animazioni dei pianeti non seguono proporzioni e velocità orbitali realistiche: sono state volutamente semplificate per garantire una migliore esperienza visiva e prestazioni più fluide.

La rappresentazione del Sistema Solare non rispetta le scale reali né per dimensioni né per distanze, in quanto l'obiettivo principale era rendere il contenuto più comprensibile e adatto a un pubblico non tecnico. L'applicazione non include effetti sonori o musiche di accompagnamento, poiché lo sviluppo si è concentrato sulle funzionalità essenziali.

Infine, la gestione degli errori relativi all'API NASA è piuttosto limitata: in caso di problemi di connessione, l'app mostra solo un semplice messaggio senza offrire un contenuto alternativo o una modalità offline.



8 Consuntivo

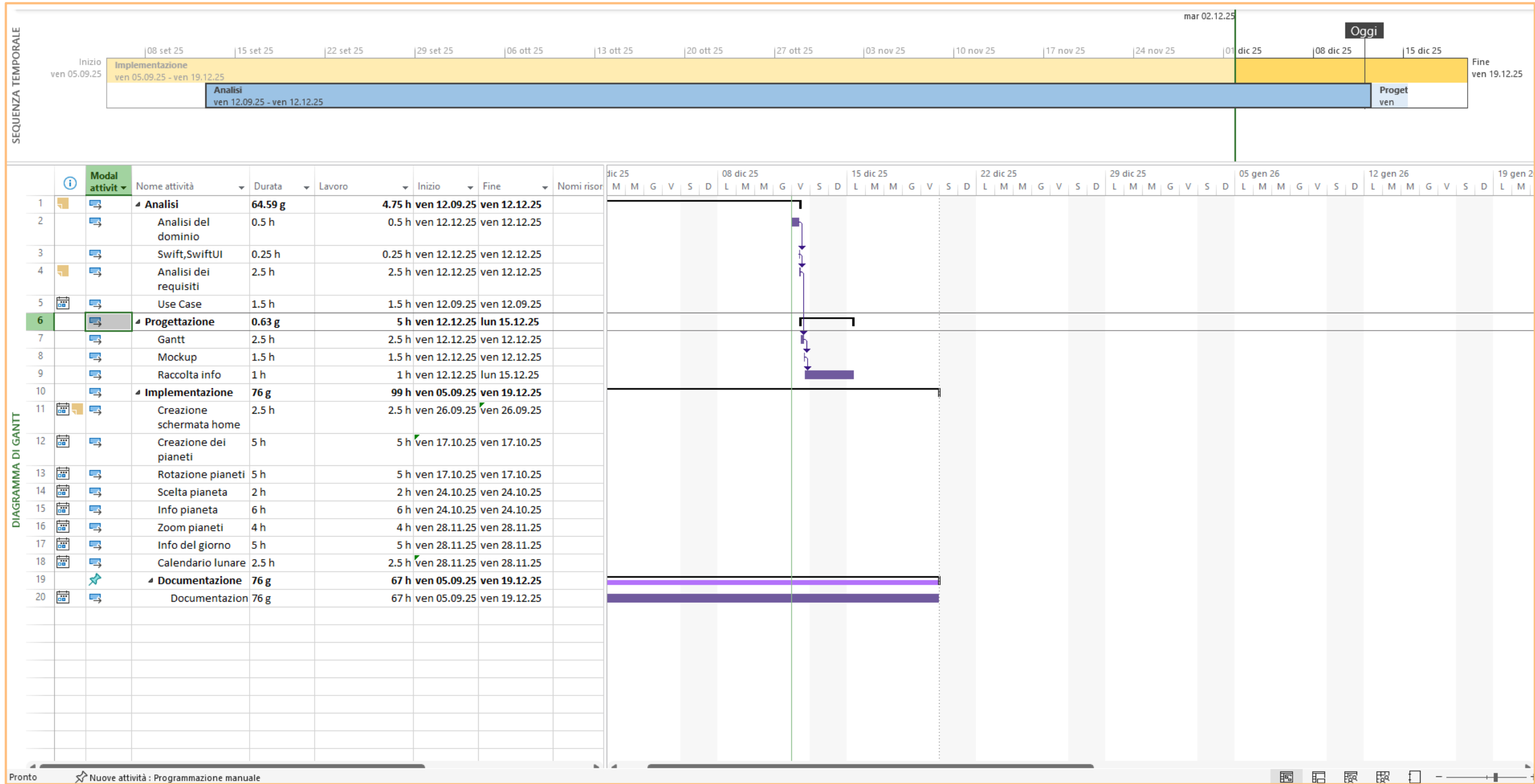


Figura 23 - Gantt Consuntivo



Il diagramma di Gantt consuntivo rappresenta l'andamento reale del progetto, mostrando le attività così come sono state effettivamente svolte nel tempo, in termini di durata, impegno orario e periodo di esecuzione.

A differenza del Gantt pianificato, il consuntivo riflette i dati reali emersi durante lo sviluppo del progetto, evidenziando eventuali scostamenti rispetto alla pianificazione iniziale.

Nel consuntivo il progetto risulta articolato nelle seguenti fasi:

8.1 Analisi

Nel consuntivo questa fase mostra una durata effettiva inferiore rispetto a quella pianificata, segno che le attività di analisi sono state svolte in modo più rapido ed efficiente del previsto, probabilmente grazie a una buona chiarezza iniziale degli obiettivi.

8.2 Progettazione

Emerge che questa fase ha richiesto meno giorni ma un impegno orario concentrato, risultando compatta e ben organizzata. Le attività sono state completate senza particolari ritardi, confermando una buona coerenza tra progettazione e requisiti.

8.3 Implementazione

Nel consuntivo l'implementazione si estende su un periodo temporale ampio, mostrando un carico di lavoro reale superiore a quello inizialmente stimato. Questo è tipico delle fasi di sviluppo, dove emergono complessità tecniche, correzioni e miglioramenti progressivi.

8.4 Documentazione

La documentazione è stata portata avanti in parallelo con le altre attività.

Nel consuntivo risulta distribuita lungo gran parte del progetto, evidenziando un approccio continuo alla stesura della documentazione tecnica e descrittiva, anziché concentrarla solo nella fase finale.

8.5 Differenze

Dal confronto tra il diagramma pianificato e quello consuntivo emergono le seguenti differenze principali:

- Durata delle attività
 - Nel pianificato, alcune fasi (in particolare analisi e progettazione) erano distribuite su un arco temporale più ampio.
 - Nel consuntivo, tali attività risultano completate in tempi più brevi, mentre l'implementazione ha richiesto più tempo e più ore di lavoro rispetto alle stime iniziali.
- Distribuzione del carico di lavoro
 - Il Gantt pianificato presenta una distribuzione più uniforme del lavoro.
 - Il Gantt consuntivo evidenzia invece un carico maggiore concentrato sulla fase di implementazione, tipico di un progetto software reale.
- Scostamenti temporali
 - Alcune attività di implementazione nel consuntivo risultano spostate rispetto alle date inizialmente previste.
 - Questi scostamenti sono dovuti a raffinamenti funzionali, debugging e adattamenti tecnici emersi durante lo sviluppo.
- Realismo della pianificazione
 - Il Gantt pianificato rappresenta una stima teorica basata su ipotesi iniziali.
 - Il Gantt consuntivo fornisce una visione realistica e concreta dell'effettivo svolgimento del progetto, risultando uno strumento fondamentale per valutare l'esperienza acquisita e migliorare future pianificazioni.

	SAMT – Sezione Informatica	Pagina 41 di 45
	Implementazione di Nebula in Swift	

8.6 Conclusione

Il confronto tra pianificato e consuntivo mostra che il progetto è stato completato con successo, nonostante alcuni scostamenti temporali.

Il Gantt consuntivo evidenzia una gestione efficace delle attività e una buona capacità di adattamento alle difficoltà emerse, fornendo una base solida per l'analisi finale e per il miglioramento delle future stime di progetto.



9 Conclusioni

9.1 Sviluppi futuri

Questo progetto rappresenta solo il punto di partenza di qualcosa che desidero far crescere e trasformare in un applicativo unico nel suo genere. Le idee per il futuro sono molte e tutte mirano a rendere l'esperienza dell'utente più ricca, immersiva e coinvolgente.

Uno dei primi miglioramenti previsti riguarda l'interazione con il Sistema Solare: vorrei rendere la funzione di ingrandimento e rimpicciolimento molto più flessibile, dinamica e realistica, così da offrire una visualizzazione più naturale e coerente con le proporzioni reali dei pianeti.

A livello di dati, un'evoluzione importante sarebbe l'integrazione di nuove API, tra cui una dedicata al calendario lunare. Questo permetterebbe alla pagina della Luna di mostrare sempre informazioni aggiornate sulle fasi, sulle posizioni e sugli eventi correlati, senza necessità di aggiornamenti manuali.

Vorrei inoltre aggiungere una sezione dedicata agli eventi celesti, come eclissi, allineamenti planetari, congiunzioni e sciami meteorici, per offrire all'utente uno strumento utile per scoprire cosa può osservare nel cielo in date specifiche.

Un'altra espansione significativa riguarda l'integrazione dell'astrologia: mi piacerebbe creare una pagina che permetta di calcolare il tema natale di una persona, combinando così l'aspetto astronomico con quello astrologico per un approccio più completo e multidisciplinare.

Infine, una delle funzionalità che considero più affascinanti è l'aggiunta di una pagina che, tramite l'uso della fotocamera del dispositivo, permetta di identificare le costellazioni nel cielo. L'idea sarebbe quella di puntare il telefono verso la volta celeste e ottenere informazioni in tempo reale sulle stelle e sulle costellazioni inquadrare, rendendo l'applicativo non solo informativo, ma anche interattivo ed esplorativo.

Nel complesso, questi sviluppi futuri mirano a costruire un applicativo che unisca scienza, tecnologia, creatività e passione, rendendola uno strumento versatile, aggiornato e capace di coinvolgere un pubblico sempre più ampio. Le possibilità sono enormi, e questo progetto rappresenta una base solida su cui far crescere un'idea ancora più ambiziosa.

9.2 Considerazioni personali

Realizzare questo progetto non è stato soltanto un esercizio didattico, ma un vero e proprio percorso di crescita personale. Mi sono messa alla prova lavorando con un linguaggio di programmazione che non avevo mai utilizzato prima, per di più molto diverso da quelli affrontati durante il mio percorso scolastico. Affrontare Swift e SwiftUI da zero ha richiesto impegno, pazienza e tanta voglia di capire, ma proprio per questo la soddisfazione finale è stata ancora più grande.

Uno degli obiettivi che mi ero posta era quello di rispettare le scadenze senza dover lavorare oltre gli orari scolastici, per verificare se fossi in grado di organizzare il tempo in modo efficace — e riuscirci mi ha reso particolarmente fiera. Questo progetto mi ha permesso di sperimentare un flusso di lavoro più simile a quello reale: pianificazione, sviluppo, revisione e consegna nei tempi previsti.

Un altro aspetto molto significativo è stato il fatto che questo fosse il primo applicativo sviluppato a scuola senza requisiti imposti dai professori. Questa libertà progettuale mi ha dato l'opportunità di esprimere la mia creatività, fare scelte autonome e costruire qualcosa che rispecchiasse davvero i miei interessi.

La scelta di un tema legato all'astronomia non è stata casuale: mi ha permesso di unire una delle mie più grandi passioni al mio percorso informatico. Lavorare su un applicativo che rappresenta gli oggetti celesti, le loro orbite e le loro caratteristiche ha reso lo sviluppo non solo formativo, ma anche profondamente piacevole.

Nonostante la difficoltà iniziale, imparare Swift si è rivelata una scelta vincente: ho acquisito competenze nuove, ho compreso modalità di sviluppo differenti rispetto a quelle a cui ero abituata e ho sperimentato un ambiente moderno e molto apprezzato nel mondo reale.

In conclusione, questo progetto non mi ha insegnato soltanto un linguaggio, ma mi ha dimostrato che sono in grado di affrontare nuove sfide, organizzare il mio lavoro in autonomia e trasformare un'idea in un'applicazione funzionante. È stato un percorso che mi ha arricchita sia dal punto di vista tecnico, sia da quello personale, e costituisce per me un piccolo ma significativo traguardo.

10 Glossario

Termine	Descrizione
API	Application Programming Interface : un insieme di funzioni e regole che permette a due software di comunicare tra loro.
API-REST	Representational State Transfer API : tipo di API che usa il protocollo HTTP (GET, POST, PUT, DELETE) per scambiare dati in modo semplice e standard.
APOD	Astronomy Picture of the Day : servizio NASA che pubblica ogni giorno un'immagine astronomica accompagnata da una breve spiegazione.
AVKit	Framework Apple che gestisce la riproduzione di video e audio.
Bundle	Contenitore che raccoglie tutte le risorse dell'app, come file, immagini e dati.
Circle	Elemento grafico di SwiftUI utilizzato per disegnare un cerchio.
Codable / Decodable	Protocollo di Swift che permette di convertire facilmente oggetti da e verso formati come JSON.
Data	Tipo Swift che rappresenta una sequenza di byte, utile per leggere file o dati ricevuti da internet.
do/catch	Struttura usata in Swift per gestire gli errori durante operazioni che possono fallire.
JSON	JavaScript Object Notation : formato leggero e molto usato per memorizzare e scambiare dati in modo strutturato.
JSONDecoder	Oggetto che converte un file o una stringa JSON in dati Swift.
Linear Animation	Animazione con velocità costante, senza cambiamenti di ritmo.
MVVM	Model-View-ViewModel : struttura che separa i dati, la logica e l'interfaccia grafica per rendere il codice più pulito e gestibile.
NASA	L'agenzia spaziale americana, fornitrice di dati e servizi come APOD.
NavigationStack	Componente SwiftUI che gestisce la navigazione tra più schermate.
Offset	Modificatore SwiftUI che sposta una vista lungo gli assi X o Y.
onAppear	Modificatore SwiftUI che permette di eseguire del codice quando una vista compare sullo schermo.
RepeatForever	Impostazione che fa ripetere un'animazione all'infinito.
RotationEffect	Modificatore SwiftUI che permette di ruotare una vista.



Safe Area	Area dello schermo non coperta da elementi come notch, barra di stato o bordi curvi.
Sheet	Finestra modale che appare sopra la vista principale per mostrare contenuti aggiuntivi.
Sky Map	Rappresentazione grafica del cielo con stelle, pianeti e altri oggetti astronomici.
Stroke	Bordo o linea che definisce il contorno di una forma grafica.
Swift	Linguaggio di programmazione principale per creare app su piattaforme Apple.
Swift Playground	Ambiente interattivo in cui è possibile scrivere ed eseguire codice Swift in modo semplice e immediato.
SwiftUI	Framework che permette di creare interfacce grafiche usando un approccio dichiarativo.
UIKit	Framework storico di Apple per creare interfacce utente su iOS in modo più tradizionale e imperativo.
URL	Uniform Resource Locator : indirizzo che indica la posizione di un file o di una risorsa nel dispositivo o sul web.
VideoPlayer	Vista SwiftUI che mostra e gestisce la riproduzione di un video tramite AVPlayer.
withAnimation	Funzione che applica animazioni ai cambiamenti di stato della UI.
ZStack	Contenitore SwiftUI che posiziona più viste una sopra l'altra.

	SAMT – Sezione Informatica	Pagina 45 di 45
	Implementazione di Nebula in Swift	

11 Bibliografia

11.1 Sitografia

- <https://api.nasa.gov/> - API NASA, 14.11.2025 – Utilizzato per APOD
- <https://apiverve.com/marketplace/moonphases> - API Verve, 28.11.2025 – Utilizzato per la fase lunare
- <https://www.w3schools.com/swift/default.asp> – W3SCHOOLS, 05.09.2025 – 19.12.2025 – Basi di Swift
- <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/guidedtour/> - Swift Programming Language, 05.09.2025 – 19.12.2025 – Basi di Swift
- <https://chatgpt.com/> - ChatGPT, 05.09.2025 – 19.12.2025 – Utilizzato nel corso del progetto (Ortografia della documentazione, Implementazione)
- <https://gemini.google.com/app?hl=it> – Gemini, 05.09.2025 – 19.12.2025 – Utilizzato nel corso del progetto (Ortografia della documentazione, Implementazione)
- <https://github.com/russell-archer/SwiftUIAPOD> - SwiftAPOD, 14.11.2025 – Insegnamento dell'API
- <https://www.google.com/imghp?hl=it&ogbl> – Google Immagini, 05.09.2025 – 19.12.2025 – Utilizzato per le immagini necessarie per il progetto
- <https://www.youtube.com/watch?v=BvXa1n9fjow&t=63s> – Venere: National Geographic – 31.10.2025 – Video informativo su Venere
- <https://www.youtube.com/watch?v=HCDVN7DCzYE> – Terra: National Geographic – 31.10.2025 – Video informativo sulla Terra
- <https://www.youtube.com/watch?v=D8pnmwOXhoY> – Marte: National Geographic – 31.10.2025 – Video informativo su Marte
- <https://www.youtube.com/watch?v=PtkqwsllbLY8> – Giove: National Geographic – 31.10.2025 – Video informativo su Giove
- <https://www.youtube.com/watch?v=epZdZaEQhS0> – Saturno: National Geographic – 31.10.2025 – Video informativo su Saturno
- <https://www.youtube.com/watch?v=m4NXbFOiOGk> – Urano: National Geographic – 31.10.2025 – Video informativo su Urano
- <https://www.youtube.com/watch?v=NStn7zZKXfE> – Nettuno: National Geographic – 31.10.2025 – Video informativo su Nettuno
- <https://www.youtube.com/watch?v=0KBjnNuhRHs> – Mercurio: National Geographic – 31.10.2025 – Video informativo su Mercurio
- https://en.wikipedia.org/wiki/Solar_System – Wikipedia Solar System, 24.10.2025 – Informazione sui pianeti
- <https://turboscribe.ai/it/downloader/youtube/mp4> – TurboScribe, 31.10.2025 – Convertitore da YouTube a MP4
- <https://moodle.edu.ti.ch/cpt/course/view.php?id=2114> – Moodle, 05.09.2025 – 19.12.2025 – Consultato il modulo 306
- <https://github.com/lindabytyqi/Sistema-solare> – GitHub Sistema Solare, 05.09.2025 – 19.12.2025 – Utilizzato per salvare il progetto
- <https://www.drawio.com/> - Drawio, 19.09.2025 – Utilizzato per la progettazione
- <https://moqups.com/it/> - Moqups, 19.09.2025 – Utilizzato per la realizzazione dei Mockup

12 Allegati

- [Diario di lavoro](#)
- [QdC](#)
- [Gantt-Pianificazione.pdf](#)
- [Gantt-Consuntivo.pdf](#)
- [README](#)
- [Nebula - Prodotto](#)